Network Analysis of Leetcode Problem Space

G Bhanu Prakash	12240620
K Aditya	12240870
B Hari Charan Goud	12240360
K Nithin	12240780
Y Suchethan Reddy	12242050

CS551 - Network Science

Indian Institute of Technology Bhilai

Abstract

Networks provide a powerful framework for analyzing complex real-world systems by representing entities as nodes and their relationships as connections. This approach has been widely used in domains such as social networks, biological systems, and information networks to uncover hidden patterns, identify influential components, and understand structural organization. Applying similar principles to algorithmic problem datasets can offer deeper insights into how concepts are interconnected and how knowledge is structured.

In this work, the LeetCode problem set is modeled as a network, where problems and their associated topic tags form a bipartite graph, along with its corresponding projections. Using network science techniques such as centrality measures, community detection, and co-occurrence analysis, the study explores relationships between problems and topics, identifies dominant concepts, and reveals underlying clusters representing algorithmic patterns.

The analysis shows that the problem space is highly structured, with a small number of core topics acting as hubs and connecting a large portion of the network. Furthermore, the network structure reflects problem difficulty and conceptual dependencies, enabling the identification of efficient learning strategies. A minimal subset of problems is also determined that covers the majority of concepts, highlighting redundancy within the dataset.

Overall, this work demonstrates how network-based modeling can provide a systematic and data-driven approach to understanding problem distributions and optimizing learning in algorithmic practice.

Contents

1 Introduction **1**

2 Data Collection and Preprocessing **3**

2.1 Data Collection 3

2.2 Dataset Description 3

2.3 Data Preprocessing 4

2.4 Data Preparation for Network Construction 4

3 Problem Tag Graph **6**

3.1 Problem–Tag Bipartite Network 6

3.1.1 Tag nodes Degree Analysis 7

3.1.2 Problem Degree Analysis 8

3.1.3 Node-Level Centrality Analysis 9

3.1.4 Analysis by Difficulty 13

3.1.5 The Set Cover Problem 17

3.1.6 Overall Structural Interpretation 19

4 Tag Tag Co-Occurence Graph **20**

4.1 Tag-Tag Co-occurrence Network 20

4.1.1 Frequent Tags (Degree Analysis) 21

4.1.2 Tag-Tag Co-occurrence Strength (Edge Weight Analysis) 22

4.1.3 Histogram of Tag Co-occurrence Frequencies 23

4.1.4 Betweenness Centrality Analysis 23

4.1.5 Community Detection (Modularity) 24

4.1.6 Tags per Community Distribution 25

4.1.7 Individual Community Analysis 26

5 Limitations **30**

5.1 30

5.1.1 Graph Scale and Problem–Problem Projection 30

5.1.2 Static Snapshot of the Dataset 30

5.1.3 Tag Granularity 31

5.1.4 Greedy Set Cover Approximation 31

5.1.5 Absence of User Behavioral Data 31

5.1.6 Bipartite Clustering Limitation 31

6 Future scope and improvements **32**

6.1 32

6.1.1 Scalable Problem–Problem Network Analysis 32

6.1.2 Weighted and Directed Network Extensions 32

6.1.3 Integration of User Performance Metrics 32

6.1.4 Dynamic Network Analysis 33

6.1.5 Personalized Learning Path Recommendation 33

6.1.6 Incorporation of Solution-Level Features 33

7 Conclusion **34**

Introduction

In recent years, network-based representations have become a powerful tool for analyzing complex systems across various domains such as social networks, biological systems, and information networks. By modeling entities as nodes and their relationships as edges, network science enables the discovery of hidden patterns, identification of influential components, and understanding of structural organization. Applying similar techniques to algorithmic problem datasets can provide valuable insights into how concepts are interconnected and how knowledge is structured.

Online coding platforms such as LeetCode host thousands of problems categorized by difficulty and associated topic tags. However, traditional approaches to problem-solving often involve solving problems sequentially or randomly, which can lead to redundancy and inefficient conceptual coverage. Many problems share similar underlying patterns, while some act as bridges connecting different domains. Without a structured approach, learners may spend significant effort on repetitive concepts while missing important connections between topics.

To address this limitation, this project models the LeetCode problem set as a network and analyzes it using tools from network science. The dataset is represented as a bipartite graph between problems and their associated tags, which is further projected into problem–problem and tag–tag networks. These representations enable the study of problem similarity, concept co-occurrence, and structural dependencies between topics.

Using various network analysis techniques such as centrality measures, community detection, and graph-based optimization, this work aims to identify key structural properties of the problem space. In particular, the analysis focuses on discovering central problems, understanding relationships between concepts, detecting clusters corresponding to algorithmic patterns, and determining a minimal subset of problems that achieves maximum conceptual coverage.

The insights derived from this study provide a data-driven perspective on problem selection

and learning strategies. By leveraging the structure of the network, learners can focus on high-impact problems and core concepts, thereby improving efficiency and reducing redundancy in preparation.

Data Collection and Preprocessing

2.1 Data Collection

The dataset used in this study is obtained from the LeetCode problem set, which contains a large collection of algorithmic problems categorized by difficulty and associated topic tags. Since the LeetCode platform dynamically renders content, automated data extraction is performed using Selenium, a browser automation tool.

The data collection process involves the following steps:

1. Load the LeetCode problem set webpage using Selenium.
2. Dynamically scroll through the page to ensure that all problems are fully rendered.
3. Extract basic information such as problem ID, title, and difficulty level.
4. Visit individual problem pages to retrieve associated topic tags.
5. Store the collected data in a structured format (JSON).
6. Convert the JSON data into CSV format for further analysis.

This automated approach ensures comprehensive coverage of the problem set and enables efficient extraction of relevant attributes required for network modeling.

2.2 Dataset Description

Each problem in the dataset is represented using the following attributes:

- **Problem ID:** Unique identifier of the problem
- **Title:** Name of the problem

- **Difficulty:** Categorized as Easy, Medium, or Hard
- **Tags:** List of associated topic tags

An example representation of a problem entry is shown below:

```
{
  "number": "1",
  "title": "Two Sum",
  "difficulty": "Easy",
  "tags": ["Array", "Hash Table"]
}
```

The final dataset consists of thousands of problems and a diverse set of tags, forming the basis for constructing the network representations.

2.3 Data Preprocessing

Before constructing the network, the dataset undergoes preprocessing to ensure consistency and usability.

The following preprocessing steps are applied:

- **Data Cleaning:** Removal of missing or incomplete entries, if any.
- **Tag Normalization:** Standardizing tag names to avoid inconsistencies (e.g., handling variations in naming conventions).
- **Duplicate Removal:** Ensuring that each problem is uniquely represented.
- **Format Conversion:** Transforming the dataset into CSV format suitable for analysis using Python libraries.
- **Tag Parsing:** Converting tag strings into structured lists for easy processing.

2.4 Data Preparation for Network Construction

To enable graph-based analysis, the preprocessed dataset is transformed into formats suitable for different network representations:

- **Bipartite Graph Input:** Problem–tag pairs are extracted to form edges between problems and tags.
- **Problem Projection:** Problem–problem relationships are generated based on shared tags.
- **Tag Projection:** Tag–tag relationships are constructed based on co-occurrence within problems.

These transformations allow the dataset to be analyzed from multiple perspectives, enabling deeper insights into problem similarity, concept relationships, and structural properties of the problem space.

The data collection and preprocessing pipeline ensures that the LeetCode problem set is transformed into a structured and analyzable format. By leveraging automated scraping and systematic preprocessing, the dataset becomes suitable for network construction and subsequent graph-based analysis. This forms the foundation for all further investigations carried out in this study.

Problem Tag Graph

3.1 Problem–Tag Bipartite Network

The constructed network represents relationships between problems and their associated tags. Each problem is connected to the set of tags that describe the concepts required to solve it. This forms a bipartite structure where nodes are divided into two disjoint sets: problems and tags.

This network captures how algorithmic concepts are distributed across problems and provides a structural representation of conceptual dependencies.

The network was constructed using the LeetCode dataset and visualized in Gephi. The final network consists of:

- Number of problem nodes: **3774**
- Number of tag nodes: **72**
- Number of edges: **11761**

Figure 3.1: Problem–Tag Bipartite Network constructed in Gephi

3.1.1 Tag nodes Degree Analysis

We analyze the degrees of each tag, which tells us how many problems contain that particular tag.

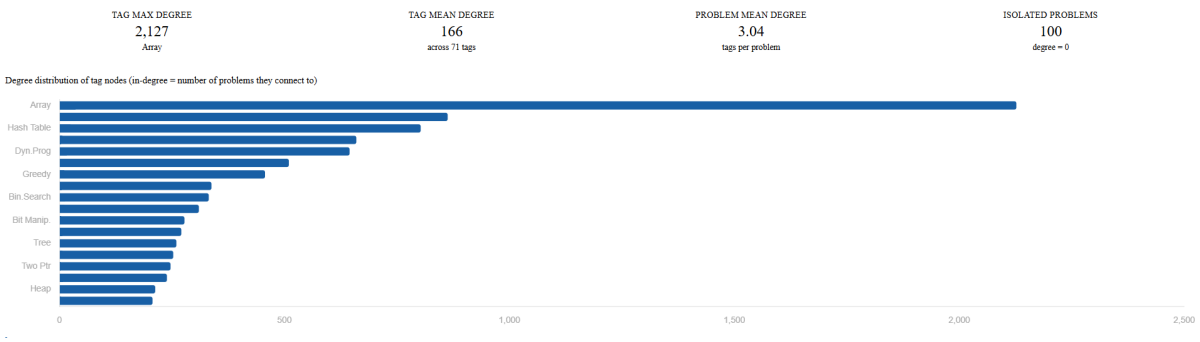


Figure 3.2: Degree distribution of tag nodes (in-degree = number of problems they connect to)

The distribution of tags follows a highly skewed pattern. A small number of tags dominate the

network, while most tags appear infrequently.

For example, the **Array** tag appears in over 2000 problems, whereas several tags occur in only one or two problems.

Tags act as hubs with high in-degree while problems are leaves with low out-degree (max 10).

Insight: The tag distribution exhibits a long-tail behavior, where a few dominant concepts cover a large portion of the problem space, while many niche tags represent specialized knowledge areas.

3.1.2 Problem Degree Analysis

The degree of a problem node represents the number of tags associated with it, which indicates the conceptual complexity of the problem.

The average number of tags per problem is approximately 3.1, with most problems having 2–3 tags. A small fraction of problems have a high number of tags (greater than 7), indicating multi-concept complexity.

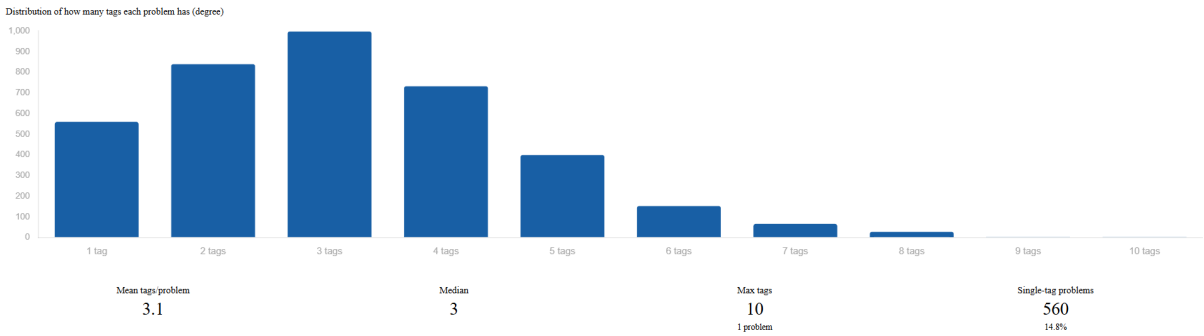


Figure 3.3: Distribution of how many tags each problem has (degree)

Insight:

- Low-degree problems correspond to single-concept problems
- High-degree problems represent complex problems involving multiple concepts

---

3.1.3 Node-Level Centrality Analysis

To analyze the importance of nodes, centrality measures such as degree and betweenness were computed.

Betweeness Centrality

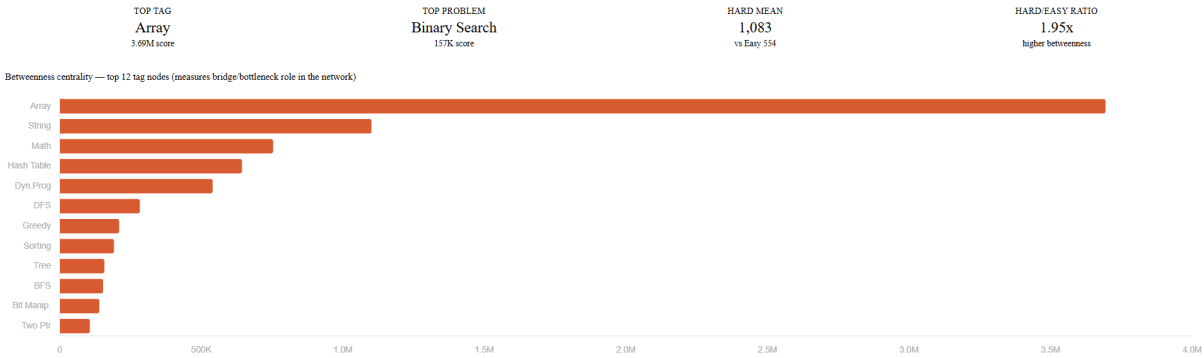


Figure 3.4: Betweenness centrality — top 12 tag nodes (measures bridge/bottleneck role in the network)

Array's betweenness (3.69M) is 3.3x higher than String (1.1M) — it is the single most critical bridge node. Removing it would fragment the graph more than removing any other node. Among problems, "Binary Search" is a surprising outlier — its simplicity of 2 tags makes it a unique bottleneck between two large clusters.

Betweenness centrality highlights tags that act as bridges between different parts of the network.

Insight:

- Tags serve as central hubs connecting multiple problems
- High betweenness nodes are critical for maintaining connectivity
- Certain problems act as bridges between different concept groups

---

Closeness and Structural Position

Closeness centrality measures how close a node is to all other nodes in the network.

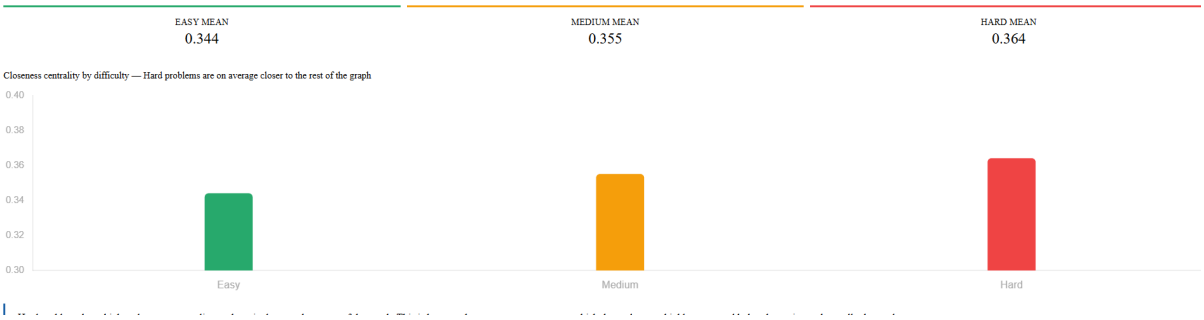


Figure 3.5: Closeness centrality by difficulty

Hard problems have higher closeness centrality — they sit closer to the centre of the graph. This is because they connect to more tags, which themselves are highly connected hubs, shortening paths to all other nodes.

Analysis shows that:

- Easy problems have lower closeness values
- Hard problems have higher closeness values

Insight: Hard problems are structurally closer to the center of the network, as they connect multiple high-degree tags and reduce path distances.

Eigenvector Centrality Analysis

Eigenvector centrality measures node influence based on the importance of its neighbors.

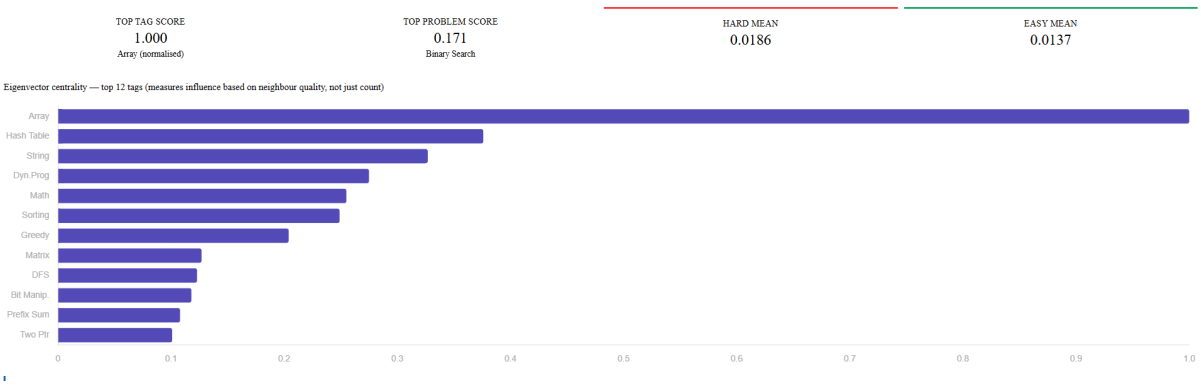


Figure 3.6: Eigenvector centrality — top 12 tags (measures influence based on neighbour quality, not just count)

It is observed that certain tags such as **Hash Table** rank higher in eigenvector centrality

compared to others with similar degree values.

Eigenvector centrality differs from degree — Hash Table (0.376) ranks above String (0.327) even though String has more problems (863 vs 803). This means Hash Table problems tend to also carry more high-degree tags, making it more "influential" in the network than raw count suggests.

Insight: Influence in the network depends not only on the number of connections but also on the importance of connected nodes.

Clustering Coefficient Analysis

The clustering coefficient measures the tendency of nodes to form triangles.

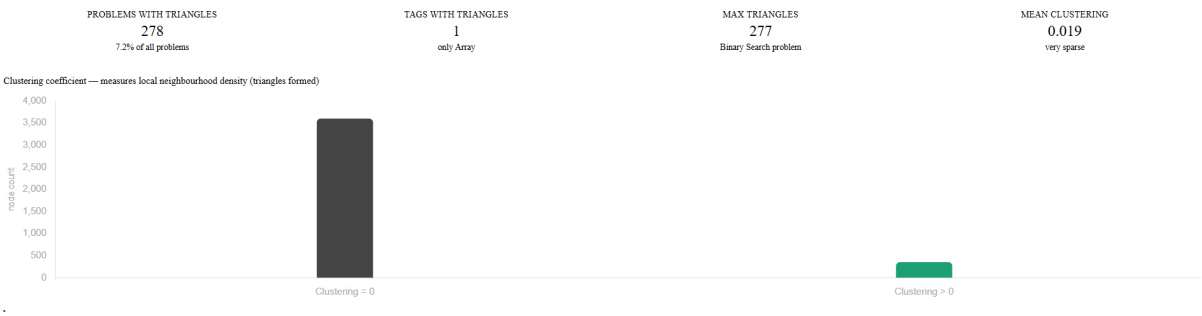


Figure 3.7: Clustering coefficient — measures local neighbourhood density (triangles formed)

- Over 90% of nodes have zero clustering coefficient
- Only a small subset of problems forms triangles

Insight: Due to the bipartite nature of the graph, clustering is minimal, as direct connections between nodes of the same type do not exist.

Community Detection (Modularity)

To uncover structural groupings within the Problem–Tag network, we applied **Louvain modularity-based community detection**. This method partitions the network into communities such that nodes within the same community are more densely connected to each other than to nodes in other communities.

In Gephi:

- Modularity was computed using the statistics panel
- Nodes were colored based on their modularity class

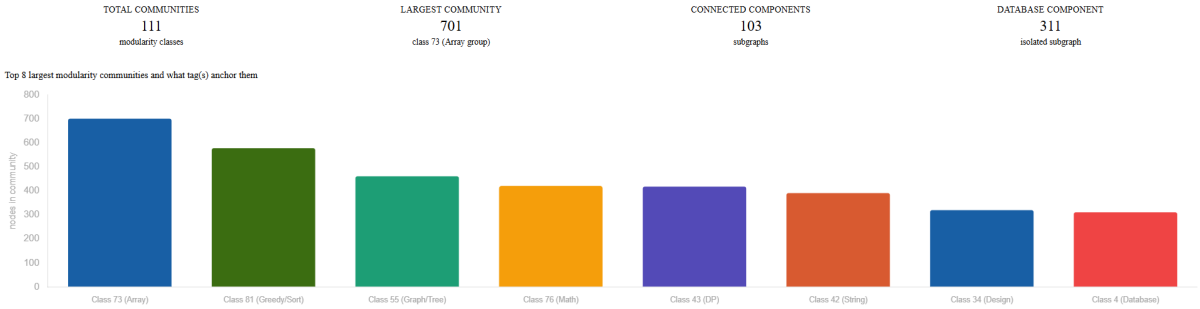


Figure 3.8: Top 8 largest modularity communities

The algorithm identified multiple communities, each representing a group of problems and tags that are closely related in terms of conceptual usage.

The Database tag forms its own isolated connected component (311 nodes) — completely disconnected from the main graph. This makes structural sense: SQL problems share no tags with algorithmic problems. The 7 largest communities each correspond to a recognisable topic cluster (Array, Greedy/Sorting, Graph/Tree, DP, String, Math, Matrix).

Insight:

- Communities correspond to clusters of related problem-solving techniques
- Core algorithmic paradigms such as Dynamic Programming, Graphs, and Greedy methods form distinct clusters
- Some problems appear near the boundaries of communities, indicating multi-concept problems

---

Eccentricity Analysis

To analyze the positional extremes of nodes in the network, we computed **eccentricity**, which measures the maximum shortest-path distance from a node to any other node in the graph.

In Gephi:

- Eccentricity was computed using the Network Diameter statistics
- Distribution of eccentricity values was analyzed for both problem and tag nodes

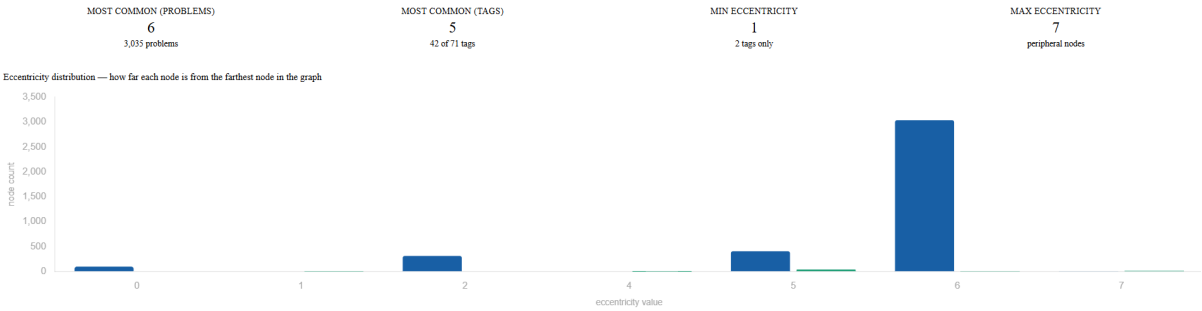


Figure 3.9: Eccentricity distribution — how far each node is from the farthest node in the graph

The results show that most nodes have eccentricity values between 5 and 6, while the maximum eccentricity observed is 7.

Insight:

- The relatively small diameter indicates a small-world structure
- Tags generally have lower eccentricity compared to problems, as they are centrally connected
- High eccentricity nodes correspond to peripheral or specialized problems

---

3.1.4 Analysis by Difficulty

Difficulty vs Degree Analysis

Definition: This analysis examines the relationship between problem difficulty and the number of associated tags (degree). The degree represents the conceptual breadth of a problem.

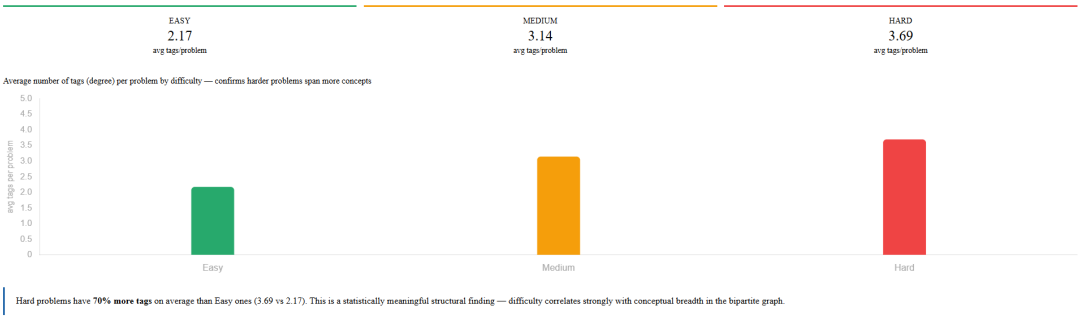


Figure 3.10: Average number of tags per problem across difficulty levels

Observation: Easy problems have an average of 2.17 tags, Medium problems have 3.14, and Hard problems have 3.69 tags. There is a clear increasing trend in degree with difficulty.

Insight: Problem difficulty is strongly correlated with conceptual complexity. Hard problems span multiple concepts and act as integrators of different algorithmic ideas.

Easy-Dominant Tags Analysis

Definition: This analysis identifies tags with the highest proportion of Easy problems, indicating beginner-friendly entry points.

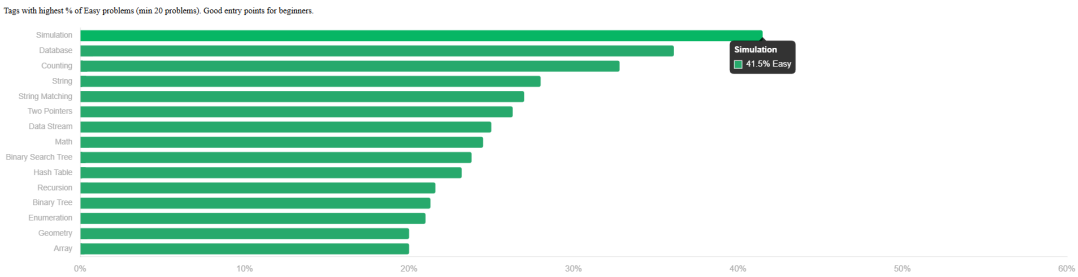


Figure 3.11: Tags with highest percentage of Easy problems

Observation: Tags such as Simulation, Database, and Counting have the highest proportion of Easy problems. Core topics like Array and Hash Table appear but with relatively lower Easy percentages.

Insight: Certain domains serve as natural starting points for beginners, while foundational topics like Arrays span across all difficulty levels and are not limited to Easy problems.

Hard-Dominant Tags Analysis

Definition: This analysis highlights tags that have the highest proportion of Hard problems, representing advanced and specialized topics.

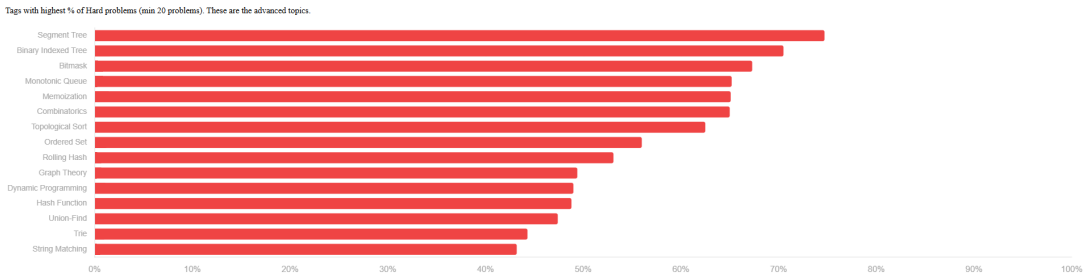


Figure 3.12: Tags with highest percentage of Hard problems

Observation: Tags such as Segment Tree, Binary Indexed Tree, and Bitmask show a high concentration of Hard problems. Dynamic Programming and Graph-related topics also appear prominently.

Insight: Advanced data structures and optimization techniques dominate the Hard difficulty level, indicating that mastery of these topics is essential for solving complex problems.

Tag Distribution by Difficulty

Definition: This analysis examines how the number of tags per problem is distributed across different difficulty levels.

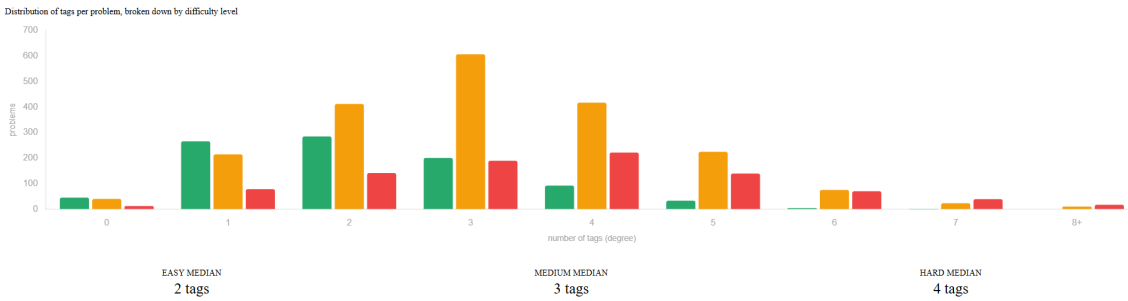


Figure 3.13: Distribution of number of tags per problem by difficulty

Observation: Easy problems are concentrated around 1–2 tags, Medium problems peak around 3 tags, and Hard problems extend towards higher tag counts (4+). Median values increase with difficulty.

Insight: Higher difficulty problems involve more concepts simultaneously, confirming that complexity arises from combining multiple techniques rather than isolated concepts.

Tag Difficulty Composition

Definition: This analysis shows the proportion of Easy, Medium, and Hard problems within each major tag.

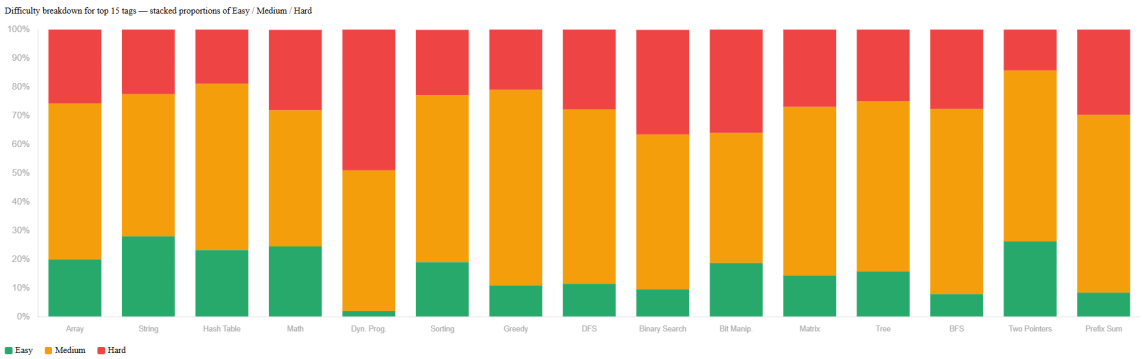


Figure 3.14: Difficulty distribution across major tags

Observation: Most tags are dominated by Medium problems. Some tags like Dynamic Programming have a high Hard proportion, while others like Two Pointers and String have more Easy problems.

Insight: Tags differ significantly in their difficulty distribution. Some act as foundational concepts, while others represent advanced domains, helping define a structured learning progression.

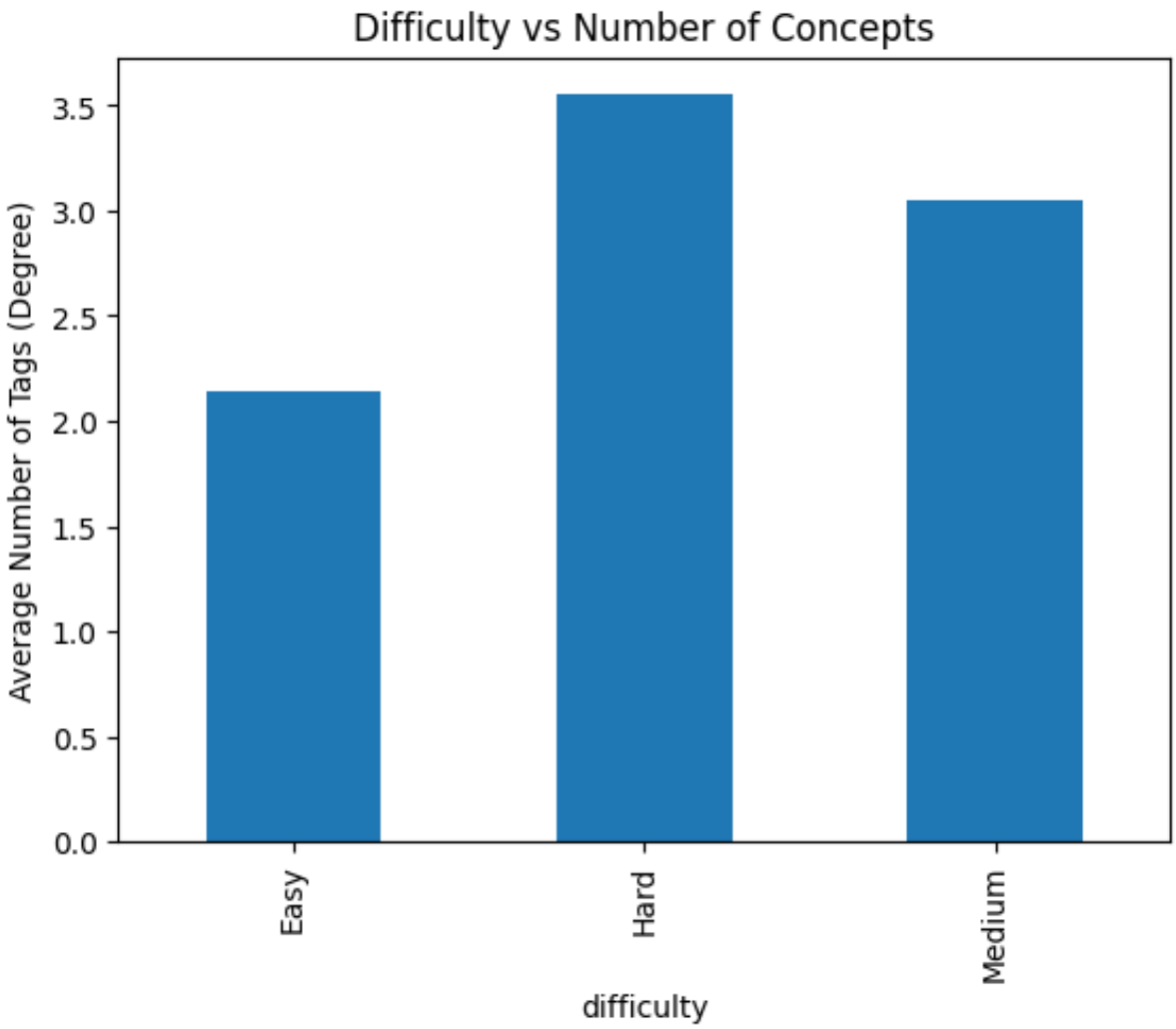


Figure 3.15: Difficulty vs Number of Concepts

3.1.5 The Set Cover Problem

Set Cover Coverage Curve

Definition: This analysis shows how tag coverage increases as more problems are selected using a greedy set cover approach across different difficulty modes.

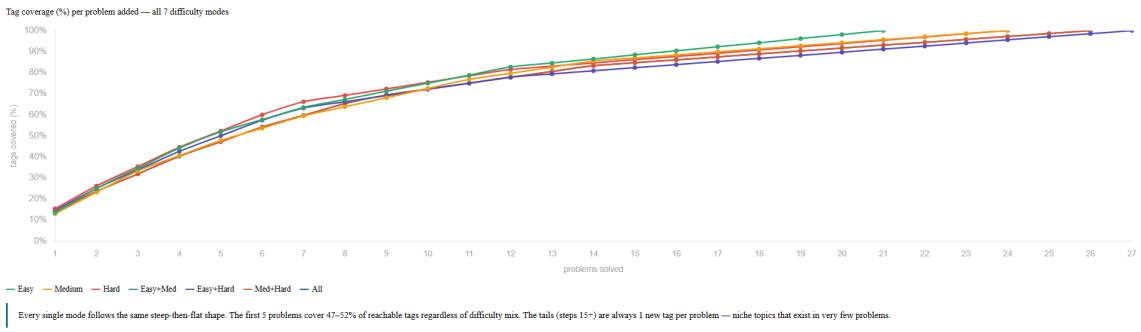


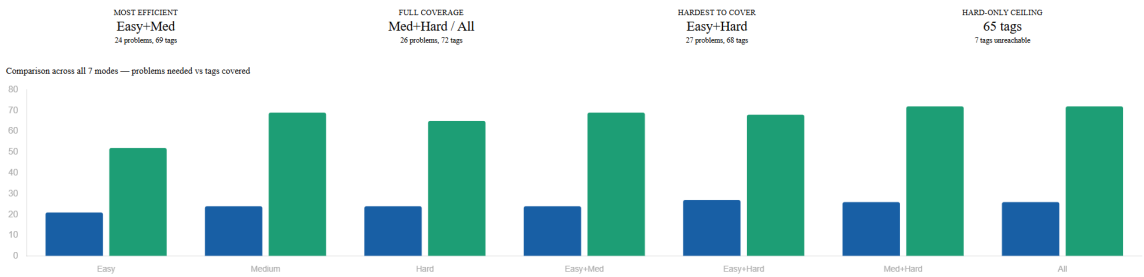
Figure 3.16: Tag coverage vs number of problems selected

Observation: All curves follow a similar pattern: rapid initial growth followed by a plateau. The first 5 problems cover nearly 50% of tags, while later additions contribute minimal new coverage.

Insight: The problem space exhibits strong redundancy. A small subset of problems provides maximum conceptual coverage, making efficient learning achievable through careful selection.

Set Cover Summary Across Difficulty Modes

Definition: This analysis compares different difficulty-based selections in terms of tags covered, number of problems required, and unreachable tags.



Mode	Pool	Tags reachable	Min problems	Unreachable tags
Easy	932	**52/72**	21	20 tags
Medium	2,027	**69/72**	24	3 tags
Hard	915	**65/72**	24	7 tags
Easy+Med	2,959	**69/72**	24	3 tags
Easy+Hard	1,847	**68/72**	27	4 tags
Med+Hard	2,942	72/72	26	None
All	3,874	72/72	26	None

Figure 3.17: Comparison of coverage efficiency across difficulty modes

Observation: Medium+Hard and All modes achieve full coverage (72/72 tags). Easy-only covers only 52 tags, while Easy+Hard struggles to reach full coverage. The minimum number of problems required for full coverage is around 26.

Insight: Medium-level problems provide the best balance between coverage and efficiency. Easy problems alone are insufficient, while combining Medium and Hard ensures complete conceptual coverage.

3.1.6 Overall Structural Interpretation

The problem–tag network exhibits several important characteristics:

- The network is highly centralized around dominant tags
- The structure is sparse with low clustering
- Difficulty correlates with structural importance
- Significant redundancy exists in the dataset

Overall Insight: The bipartite representation provides a clear understanding of how problems are structured around concepts, enabling efficient learning strategies through minimal problem selection.

Tag Tag Co-Occurence Graph

4.1 Tag-Tag Co-occurrence Network

The constructed network represents relationships between tags based on their co-occurrence across problems. Each node corresponds to a unique tag, and an edge is formed between two tags if they appear together in at least one problem. The weight of an edge indicates the number of problems in which the corresponding pair of tags co-occur.

This network captures the structural relationships between different problem-solving concepts and provides a data-driven view of how topics interact in practice.

The network was constructed using extracted problem-tag data and visualized in Gephi. The final network consists of:

- Number of nodes (tags): **N**
- Number of edges (co-occurrences): **E**

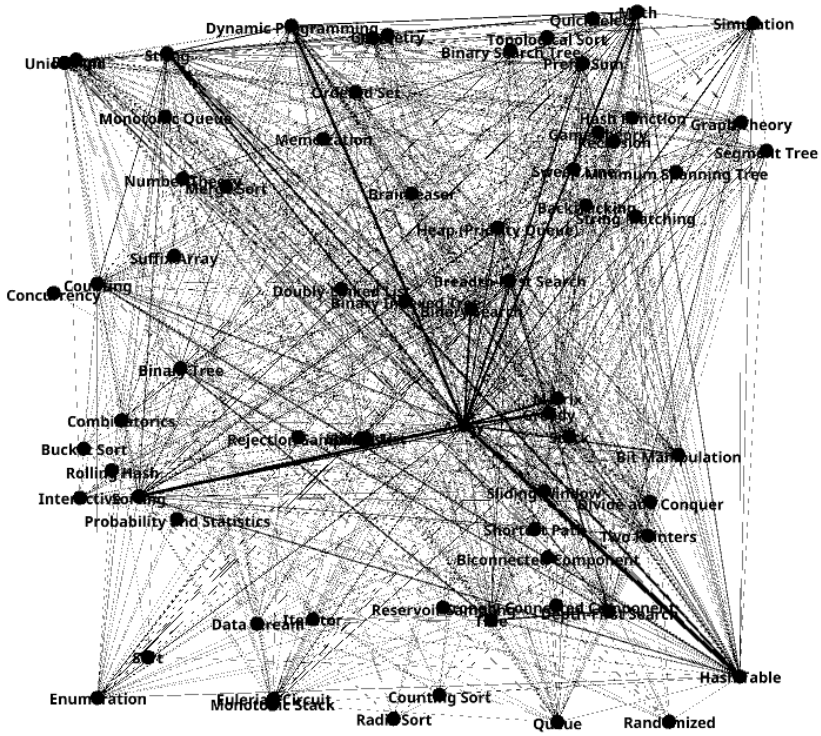


Figure 4.1: Tag-Tag Co-occurrence Network constructed in Gephi

4.1.1 Frequent Tags (Degree Analysis)

To identify the most frequently occurring and widely connected tags, we used **degree centrality**. The degree of a node represents the number of unique tags it co-occurs with, which serves as a proxy for how broadly a concept is used across problems.

In Gephi:

- Degree was computed using the statistics panel
- Node sizes were scaled according to degree values
- Additional coloring was applied to enhance visual distinction

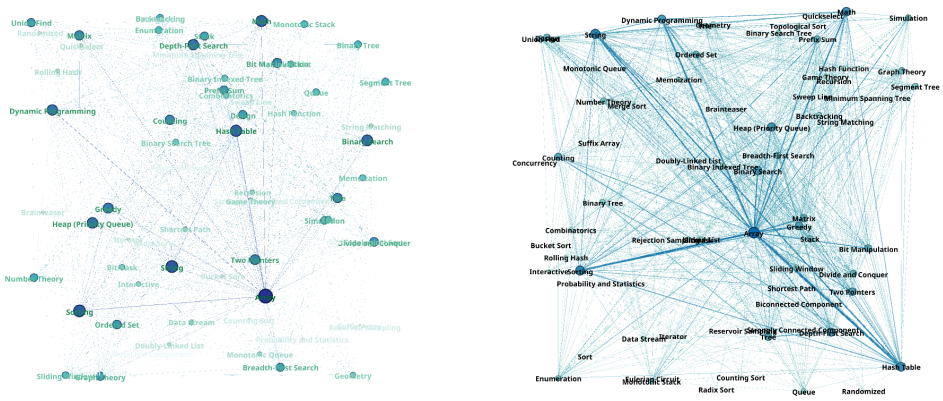


Figure 4.2: Visualization of tag frequency using degree centrality

Insight: Tags with high degree act as central concepts that connect multiple topics. These represent foundational techniques that are widely applicable across different problem types.

4.1.2 Tag-Tag Co-occurrence Strength (Edge Weight Analysis)

To analyze the strength of relationships between tags, we used **edge weights**, where each weight represents how frequently two tags appear together in problems.

In Gephi:

- Edge weights were used as a ranking attribute
- Edge thickness was scaled proportionally to weight

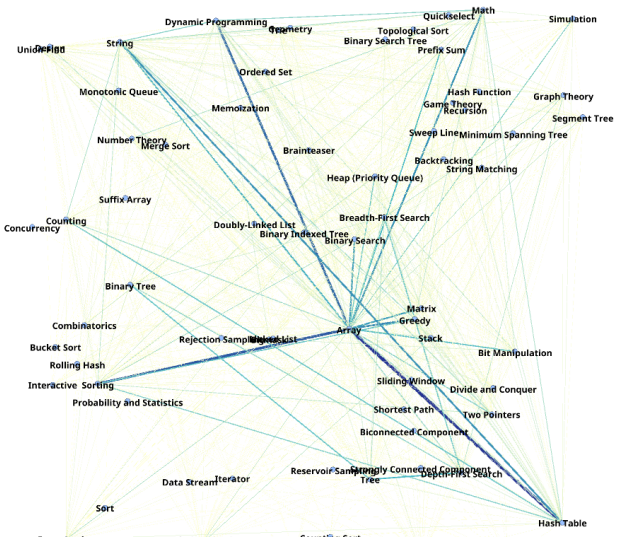


Figure 4.3: Edge weights representing tag co-occurrence frequency

Insight: Strong edges indicate pairs of concepts that frequently co-occur, suggesting strong conceptual relationships and commonly combined techniques in problem-solving.

4.1.3 Histogram of Tag Co-occurrence Frequencies

To better understand the distribution of co-occurrence strengths, a histogram of edge weights was generated.

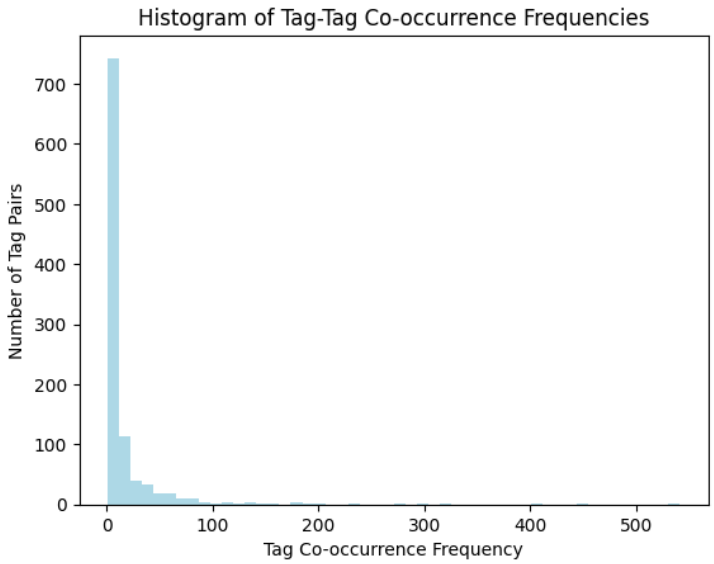


Figure 4.4: Histogram of tag co-occurrence frequencies

Insight: The histogram reveals a skewed distribution where most tag pairs occur infrequently, while a small number of pairs occur very frequently. This indicates that while many topic combinations exist, only a few dominate in practice.

4.1.4 Betweenness Centrality Analysis

To identify tags that serve as connectors between different groups of topics, we computed **betweenness centrality**. This measure quantifies how often a node lies on the shortest paths between other nodes.

In Gephi:

- Betweenness was computed using the Network Diameter function

- Node sizes were scaled based on betweenness values

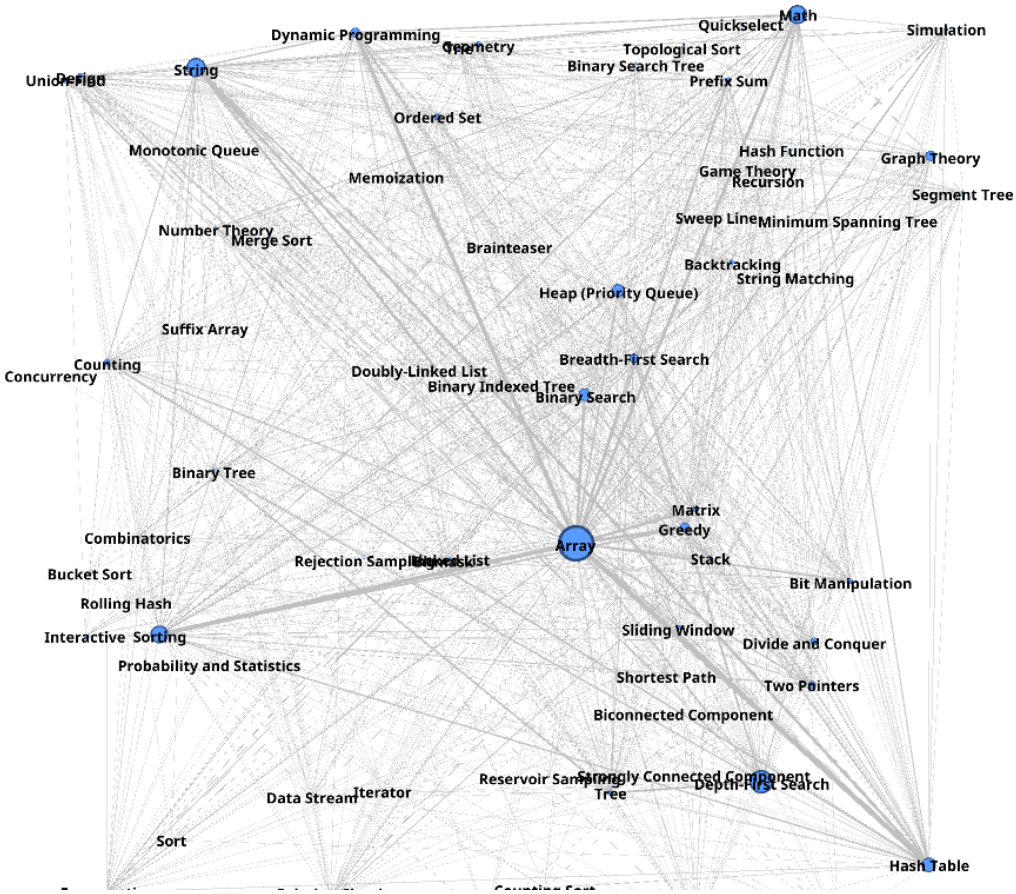


Figure 4.5: Betweenness centrality of tags

Insight: Tags with high betweenness act as bridges between different clusters, indicating concepts that connect multiple domains and enable transitions between different problem-solving techniques.

4.1.5 Community Detection (Modularity)

To uncover structural groupings in the network, we applied **Louvain modularity-based community detection**. This method partitions the network into clusters such that nodes within a cluster are more densely connected to each other than to nodes outside the cluster.

In Gephi:

- Modularity was computed using the statistics panel
- Nodes were colored according to modularity class

A total of **5 communities** were identified.

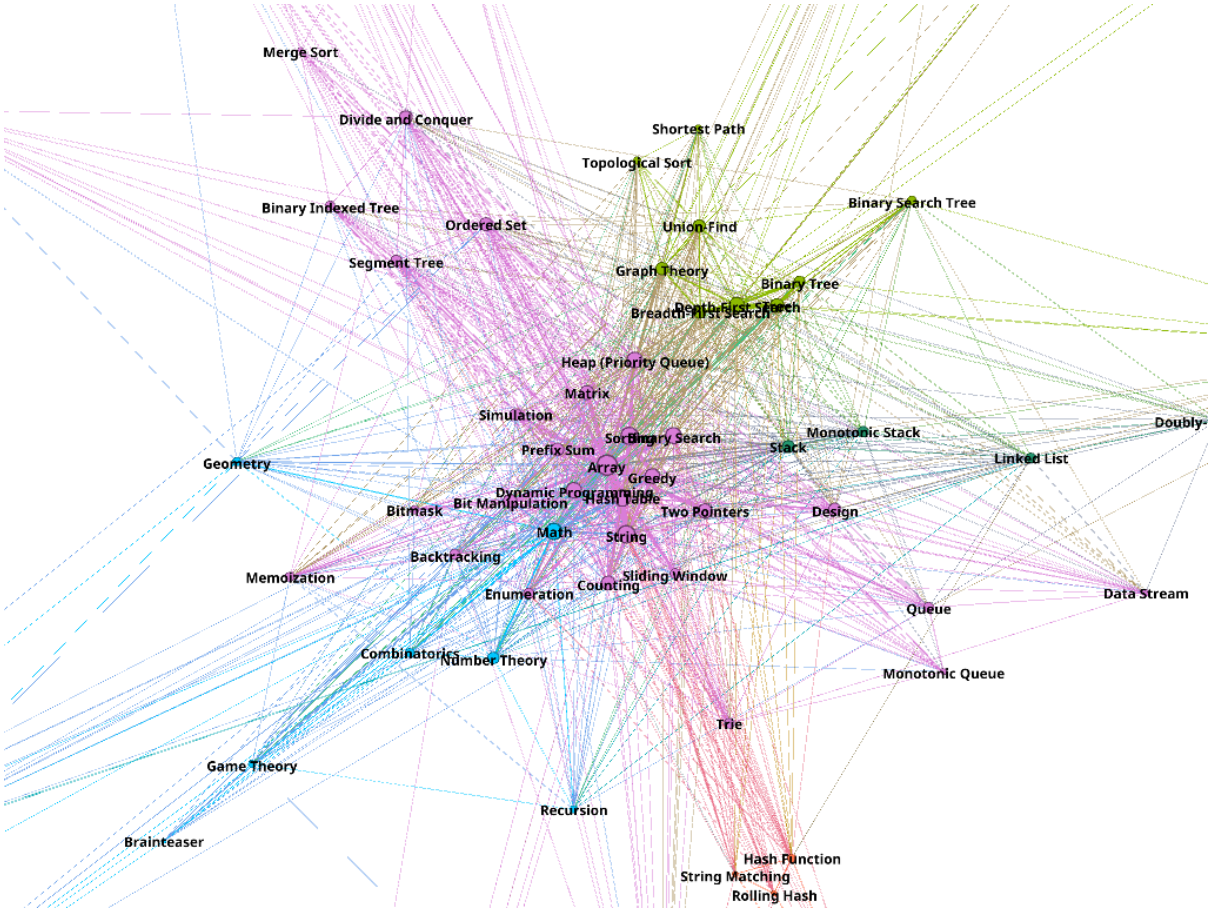


Figure 4.6: Communities detected in the tag network

Insight: Each community represents a group of closely related concepts, indicating that the problem space naturally organizes into distinct thematic clusters.

4.1.6 Tags per Community Distribution

The distribution of tags across communities was analyzed to understand the relative size of each cluster.

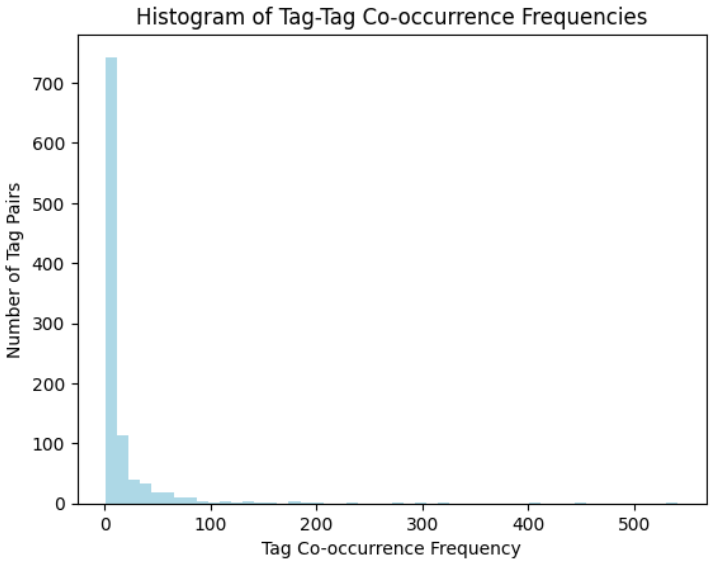


Figure 4.7: Distribution of tags across communities

Insight: Larger communities correspond to dominant and widely used topic groups, while smaller communities represent niche or specialized areas.

4.1.7 Individual Community Analysis

Each community was analyzed individually to understand its conceptual composition.

Community 0

Rolling Hash, Suffix Array, String Matching, Hash Function

Insight: This community is centered around string processing and hashing techniques, indicating problems related to pattern matching and text algorithms.

Community 1

Backtracking, Dynamic Programming, Array, Binary Search

Insight: This represents a core problem-solving cluster combining fundamental data structures with algorithmic paradigms, forming a major portion of typical problems.

Community 2

Recursion, Math, Probability, Combinatorics

Insight: This cluster corresponds to mathematically intensive problems, often involving counting, probability, and recursive formulations.

Community 3

Union-Find, Topological Sort, BFS, DFS, MST, BST

Insight: This community captures graph-based algorithms and tree structures, representing connectivity, traversal, and optimization problems.

Community 4

Linked List, Doubly Linked List, Stack, Monotonic Stack

Insight: This cluster focuses on linear data structures and stack-based techniques, often used in sequential processing problems.

Cross-Community Observations

While each community represents a distinct conceptual domain, there exist important relationships across communities. Certain tags act as bridges between clusters, indicating that real-world problems often require combining multiple paradigms. For example, dynamic programming concepts may interact with graph-based techniques, and mathematical reasoning may complement combinatorial or recursive approaches.

This interconnectivity suggests that the problem space is not strictly partitioned, but rather organized into overlapping conceptual regions.

Structural Interpretation of the Network

The overall structure of the tag-tag network reveals several important characteristics:

- The presence of dense communities indicates strong thematic grouping of concepts.
- High-degree nodes represent widely applicable techniques that appear across multiple domains.
- High-weight edges highlight frequently co-occurring concept pairs, forming common problem patterns.

- Bridge nodes (high betweenness) connect otherwise distinct clusters, enabling transitions between topics.

This demonstrates that the problem space is both *modular* and *interconnected*, which is a typical property of complex networks.

Implications for Problem Solving

The observed network structure has direct implications for learning and problem-solving strategies:

- Core communities (such as those containing arrays, dynamic programming, and graph algorithms) should be prioritized, as they cover a large portion of problems.
- Strongly connected tag pairs indicate commonly used combinations of techniques, suggesting that these concepts should be studied together.
- Smaller communities represent specialized areas, which can be approached after mastering foundational topics.
- Bridge concepts are particularly important, as they enable solving problems that span multiple domains.

Summary of Findings

From the network-based analysis, we observe that:

- The tag space is structured into a limited number of meaningful communities.
- A small number of tags act as central hubs in the network.
- Most tag combinations are rare, while a few combinations dominate problem construction.
- The organization of tags reflects practical problem-solving patterns rather than arbitrary categorization.

Overall Insight: The tag-tag network provides a structured representation of the problem space, revealing how different algorithmic concepts are organized, interconnected, and utilized in practice. This enables a deeper understanding of problem composition beyond individual topics.

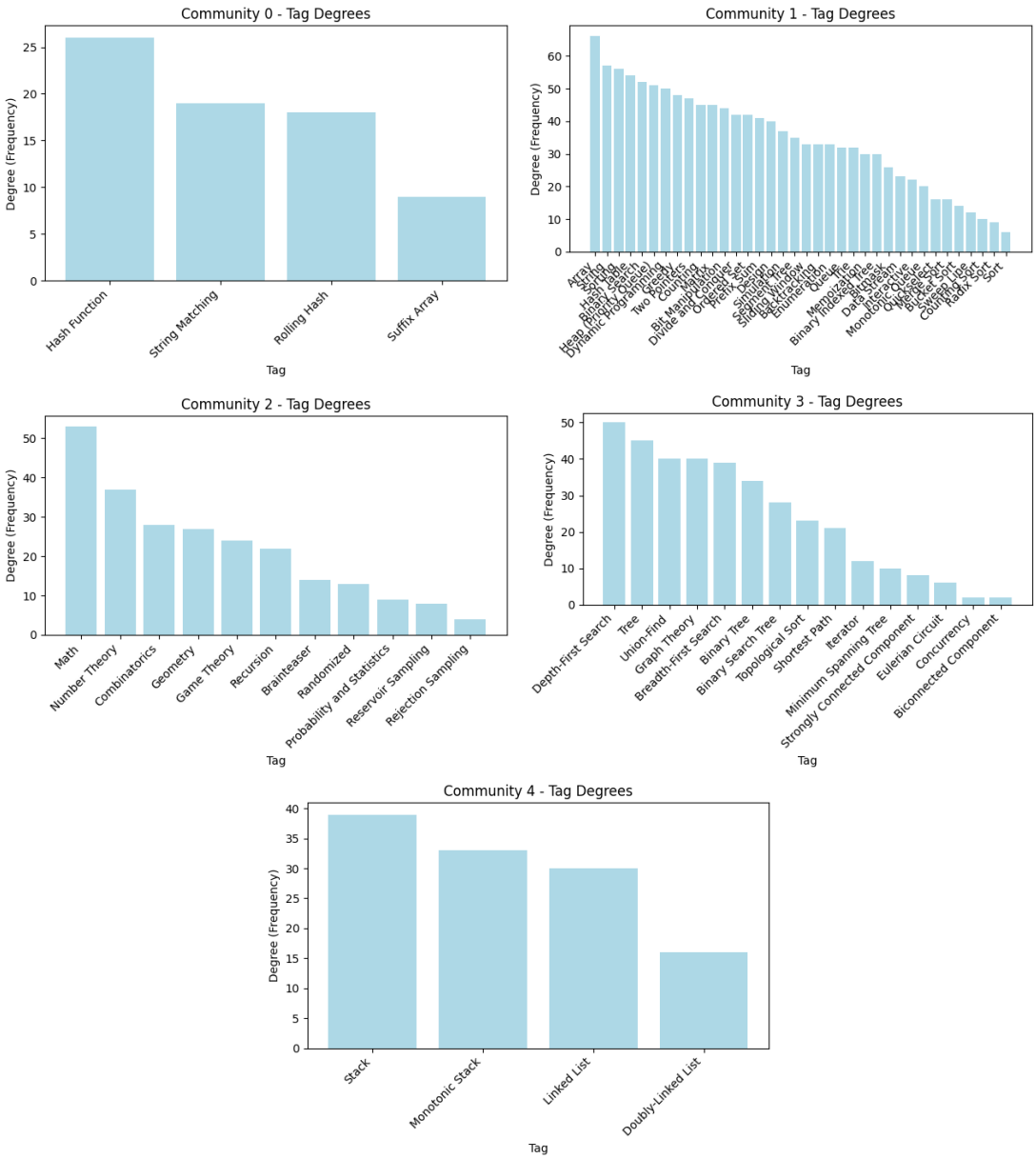


Figure 4.8: Histograms for individual communities

Insight: The decomposition of the network into these communities highlights that the problem space is not random but structured into well-defined conceptual domains.

Limitations

5.1

While this study provides meaningful insights into the LeetCode problem space, several limitations should be acknowledged.

5.1.1 Graph Scale and Problem–Problem Projection

One of the initially planned analyses was the construction of a problem–problem projection network, where two problems are connected if they share one or more common tags. However, this analysis was not feasible within the scope of this project.

The dataset contains 3,774 problem nodes, and projecting this into a problem–problem graph results in a very large number of edges, potentially in the order of hundreds of thousands. This exceeds the practical rendering and computational limits of Gephi for interactive analysis. As the size and density of the network increase, Gephi experiences significant performance degradation, making layout computation, modularity detection, and visualization unreliable.

As a result, this analysis was excluded. Future work using more scalable graph processing tools may enable exploration of this projection and reveal deeper insights into problem similarity.

5.1.2 Static Snapshot of the Dataset

The dataset used in this study represents a snapshot collected at a single point in time using automated scraping. The LeetCode platform is continuously updated with new problems and revised tags. Therefore, the results reflect the state of the dataset at the time of collection and may not fully generalize to future updates.

5.1.3 Tag Granularity

The analysis relies on topic tags assigned by the platform. These tags are manually curated and may not always capture the complete conceptual complexity of a problem. Some problems may be tagged incompletely or inconsistently, introducing noise into the network structure. Although basic normalization was performed during preprocessing, inconsistencies in tagging remain a potential source of error.

5.1.4 Greedy Set Cover Approximation

The set cover problem is computationally hard, and the greedy algorithm used in this study provides an approximate solution rather than an optimal one. The identified subset of problems, approximately 26 for full tag coverage, represents a near-optimal solution but not necessarily the minimum possible. Despite this, the greedy approach is widely used due to its efficiency and strong approximation guarantees.

5.1.5 Absence of User Behavioral Data

The analysis is based solely on structural properties of the network. It does not incorporate user-level data such as solution time, success rate, or learning progression. Including such information could provide a more comprehensive understanding of problem difficulty and practical learning outcomes.

5.1.6 Bipartite Clustering Limitation

The low clustering coefficient observed in the bipartite network is a consequence of its structure. In bipartite graphs, connections only occur between nodes of different types, preventing the formation of triangles. As a result, standard clustering metrics are not directly applicable and must be interpreted with caution.

Future scope and improvements

6.1

Several directions can extend and enhance the findings of this study.

6.1.1 Scalable Problem–Problem Network Analysis

The problem–problem projection network remains an important component that could not be fully explored in this work. Future analysis can utilize scalable graph processing frameworks such as NetworkX with sparse representations, graph databases such as Neo4j, or distributed systems such as Apache Spark GraphX to handle large-scale networks. Studying this projection can reveal clusters of structurally similar problems, identify redundant problem groups, and support the development of improved recommendation systems.

6.1.2 Weighted and Directed Network Extensions

The current network representations are primarily undirected, with weights only applied in co-occurrence graphs. Future work can extend this model by introducing directed edges to capture relationships such as conceptual dependencies between tags. This would enable the construction of a learning dependency graph, which could be used to derive structured and optimized learning sequences.

6.1.3 Integration of User Performance Metrics

The present analysis is based solely on structural relationships within the dataset. Incorporating user-level metrics such as acceptance rates, average solution times, and submission behavior can enrich the network with performance-based attributes. This would allow for more accurate modeling of problem difficulty and more effective learning path recommendations.

6.1.4 Dynamic Network Analysis

The LeetCode platform evolves continuously with the addition of new problems and updates to existing tags. Analyzing how the network changes over time can provide insights into emerging trends in algorithmic problem solving. Tracking variations in tag frequencies, co-occurrence patterns, and community structures can offer a longitudinal perspective on the evolution of the problem space.

6.1.5 Personalized Learning Path Recommendation

The results obtained from set cover optimization and community detection can be extended to build a recommendation system for learners. Given a user's current knowledge profile, defined by the set of tags already covered, the system can suggest problems that maximize new conceptual coverage in an efficient manner.

6.1.6 Incorporation of Solution-Level Features

In addition to tags, the structure of accepted solutions contains valuable information. Future work can incorporate code-level features such as solution patterns, code similarity, or abstract syntax tree representations to build a more comprehensive model of problem relationships. This would enable a multi-layered analysis combining conceptual and implementation perspectives.

Conclusion

This study presents a network-based analysis of the LeetCode problem space by modeling the dataset as a bipartite graph between problems and topic tags, and extending it to a tag–tag co-occurrence network. By applying centrality measures, community detection, degree analysis, and graph-based optimization techniques, several meaningful structural properties of the problem space are identified.

The network exhibits hub-like behavior, where a small number of dominant tags, such as Array, Hash Table, String, and Dynamic Programming, connect a large portion of the problem set. Problem difficulty shows a clear structural pattern within the network. Hard problems tend to occupy more central positions and are associated with a larger number of highly connected tags, indicating that difficulty arises from conceptual breadth rather than isolated complexity.

The set cover analysis demonstrates that a small subset of problems can provide comprehensive conceptual coverage. Specifically, 26 carefully selected problems are sufficient to cover all 72 unique tags, with the first few problems contributing a significant portion of the total coverage. This highlights the high level of redundancy in the dataset and suggests that efficient learning strategies can be designed through structured problem selection.

Community detection on the tag–tag network reveals five well-defined clusters corresponding to major algorithmic domains. These include string processing and hashing, core data structures with algorithmic paradigms, mathematical and combinatorial reasoning, graph and tree algorithms, and linear data structures involving stack-based techniques. The Database tag forms an isolated subgraph, indicating its structural separation from algorithmic problems. Additionally, tags such as Array and Greedy exhibit high betweenness centrality, emphasizing their role as

connectors between different conceptual domains.

Overall, this work demonstrates that network science provides a systematic and data-driven framework for analyzing the structure of algorithmic problem sets. The insights obtained from this approach can be used to design more efficient and informed learning strategies for competitive programming and technical interview preparation.